

Why these two features exist

When loading files into Snowflake using COPY INTO, **bad data is inevitable** (type mismatches, extra delimiters, overlength strings, malformed rows).

Snowflake gives you **two control levers**:

1. **ON_ERROR** → *Load data anyway, but control what to skip*
2. **VALIDATION_MODE** → *Don't load anything; just inspect errors*

Think of it like this:

Goal	Use
"Load what's good, ignore bad"	ON_ERROR
"Check data quality before loading"	VALIDATION_MODE

1 ON_ERROR — What it is (Concept)

ON_ERROR tells Snowflake **what to do when it encounters an error during load**.

Default behaviour

ON_ERROR = ABORT_STATEMENT

- First error → **entire COPY fails**
- **No rows loaded**

This is **unsafe for production**.

2 ON_ERROR — All Options Explained (Theory + Behavior)

♦ 1. ABORT_STATEMENT (Default)

```
COPY INTO emp
```

```
FROM @stage/employee/
```

```
ON_ERROR = ABORT_STATEMENT;
```

What Snowflake does internally

- Reads rows sequentially
- Encounters first bad row
- **Stops immediately**

- Rolls back entire load

Output

- Rows loaded: 0
- Error message shown

When to use

- DEV / unit testing
- Strict regulatory pipelines

Why NOT production

- One bad row blocks entire day's data

♦ 2. CONTINUE (Most used in PROD)

COPY INTO emp

FROM @stage/employee/

ON_ERROR = CONTINUE;

Internal behavior

- Valid rows → inserted
- Invalid rows → skipped
- Errors → logged in metadata

Output

- Partial load succeeds
- Status: PARTIALLY_LOADED

Real-time usage

- Daily batch loads
- Business data must be available

Key point

Data availability > perfect data

♦ 3. SKIP_FILE

COPY INTO emp

FROM @stage/employee/

ON_ERROR = SKIP_FILE;

Internal behavior

- If **any error** occurs in a file
- Entire file is skipped

Output

- 0 rows from that file
- Other files still load

When to use

- One file = one business unit
- Don't want partial files

◆ 4. SKIP_FILE_<num>

ON_ERROR = SKIP_FILE_5;

Meaning

- Skip file if **errors $\geq$ 5**

Why this exists

- Allows small error tolerance
- Avoids skipping files for 1–2 bad rows

Example

- 2 errors → file loads
- 5 errors → file skipped

◆ 5. SKIP_FILE_<percent>%

ON_ERROR = SKIP_FILE_0.1%;

Meaning

- Skip file if error rate > 0.1%

Production reality

- Used for large files (millions of rows)
 - % based logic scales better than row count
-

3 What happens to rejected rows with ON_ERROR?

👉 They are NOT stored automatically

Rejected rows:

- Exist only in **COPY execution metadata**
 - Must be captured manually using:
 - `VALIDATE()` (recommended)
 - `RESULT_SCAN()` (for validation runs)
-

4 VALIDATION_MODE — What it is (Concept)

`VALIDATION_MODE` is **pre-load analysis**.

“Scan the file, tell me what would fail — but don’t load anything.”

Key Rule

🚫 No data is inserted into the table

5 VALIDATION_MODE Options

◆ 1. RETURN_ERRORS

`COPY INTO emp`

`FROM @stage/employee/emp.csv`

`VALIDATION_MODE = RETURN_ERRORS;`

What Snowflake does

- Reads file
- Returns only error rows
- Stops after finding errors

Output

- Error rows + messages

Use case

- Quick debugging
- Developer testing

♦ 2. RETURN_ALL_ERRORS (Most powerful)

COPY INTO emp

FROM @stage/employee/

VALIDATION_MODE = RETURN_ALL_ERRORS;

Internal behavior

- Scans entire file
- Collects **every error**
- No data loaded

Output

- Full error report:
 - Row data
 - Column
 - Error reason
 - File name

Used in

- UAT
 - Data quality certification
 - Compliance checks
-

♦ 3. RETURN_<n>_ROWS

VALIDATION_MODE = RETURN_10_ROWS;

Meaning

- Return first 10 problematic rows

Why

- Sample inspection
- Faster execution on huge files

6 VALIDATION_MODE vs ON_ERROR (Critical Comparison)

Aspect	ON_ERROR	VALIDATION_MODE
Loads data	✓ Yes	✗ No
Used in PROD	✓ Yes	✗ Mostly
Error preview	Limited	Full
Performance	Fast	Slower
Data availability	High	None

7 How to Store Errors (PRODUCTION-MANDATORY)

◆ After ON_ERROR = CONTINUE

```
SELECT *  
FROM TABLE(VALIDATE(emp, job_id => '_last'));
```

What this does

- Reads rejected rows
- Uses COPY job metadata
- Returns error rows

◆ Persist errors

```
CREATE OR REPLACE TABLE emp_err_log AS  
SELECT *  
FROM TABLE(VALIDATE(emp, job_id => '_last'));
```

Now you have

- Permanent error audit
 - File-level traceability
 - Compliance proof
-

8 RESULT_SCAN vs VALIDATE (Don't confuse)

Function	Used with
RESULT_SCAN()	VALIDATION_MODE
VALIDATE()	ON_ERROR

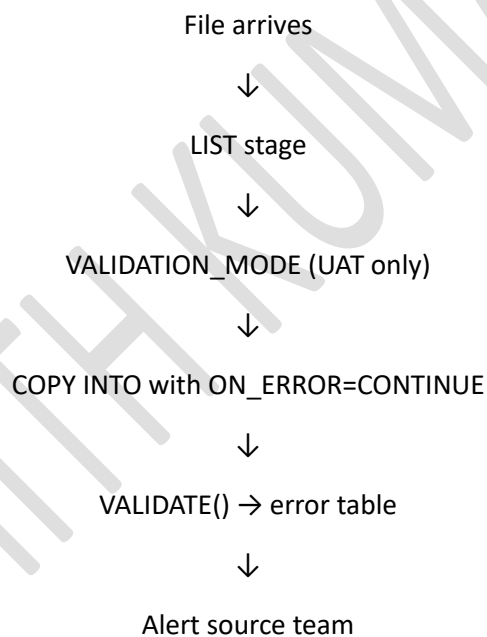
Example

SELECT *

FROM TABLE(RESULT_SCAN(last_query_id()));

Used only when VALIDATION_MODE is executed.

9 Typical Production Decision Tree



10 Common Errors & Why They Happen

Error	Root cause	Fix
Numeric conversion error	Bad data	CAST / STRING
Column mismatch	Extra delimiter	FIELD_OPTIONALLY_ENCLOSED_BY
String too long	Length exceeded	TRUNCATECOLUMNS
Date error	Format mismatch	TO_DATE / VARCHAR

1 1 Best-Practice Settings (Production-Ready)

```
COPY INTO emp
FROM @stage/employee/
FILE_FORMAT = my_csv_format
ON_ERROR = CONTINUE
TRUNCATECOLUMNS = TRUE;

And always:

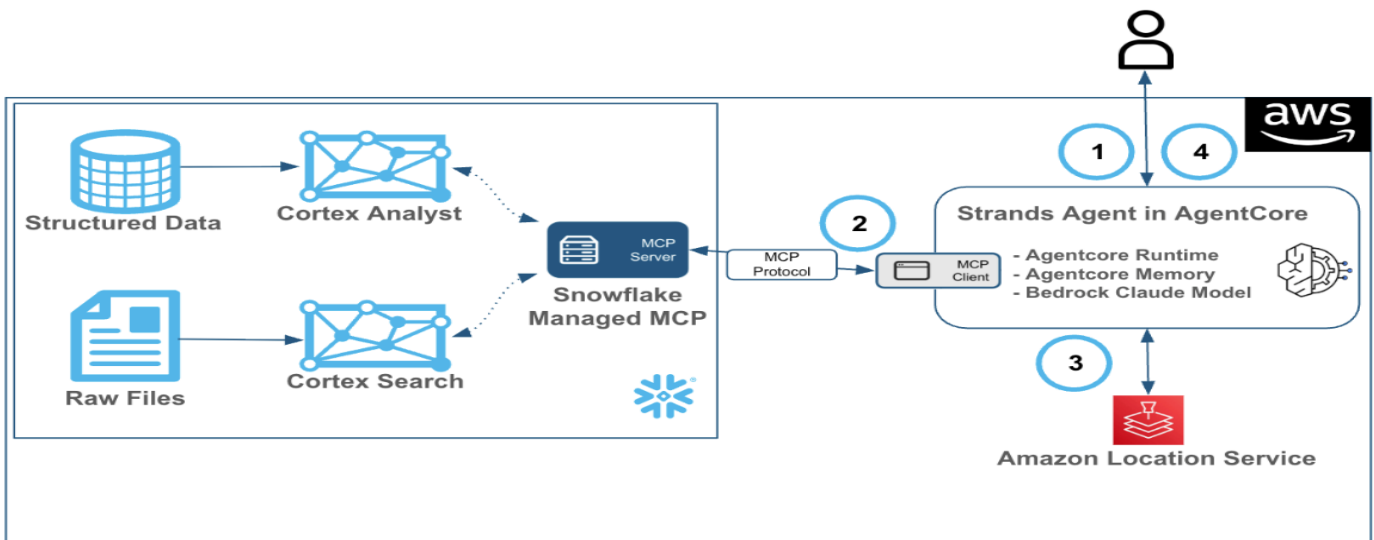
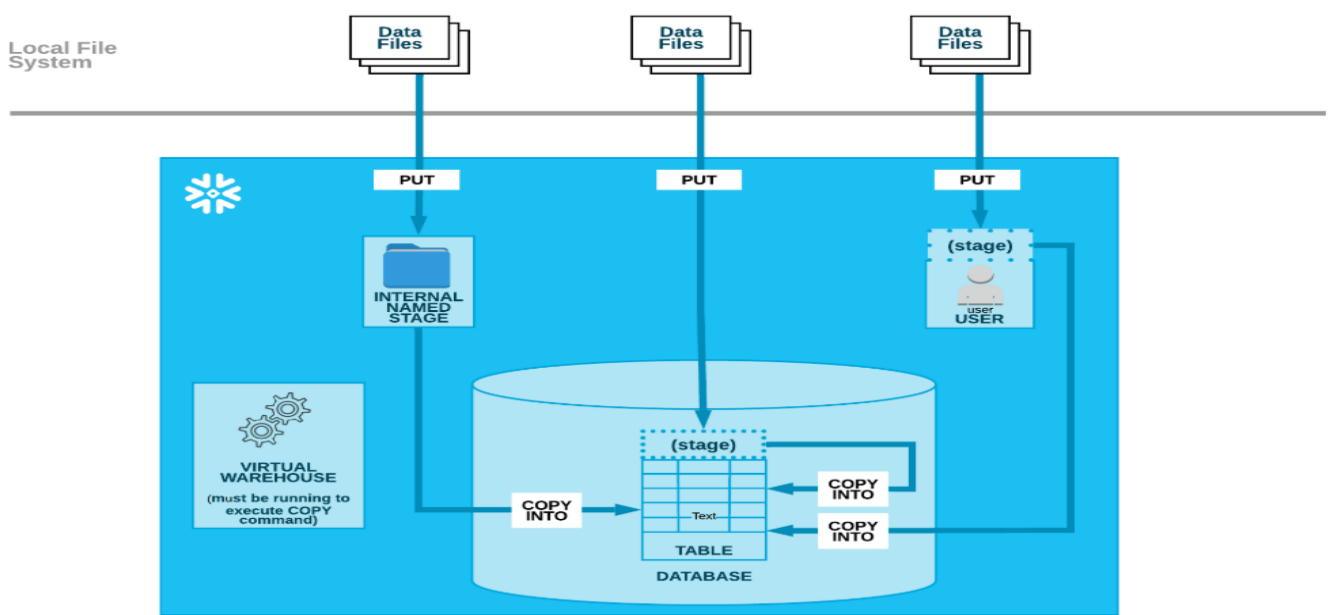
INSERT INTO emp_err_log
SELECT * FROM TABLE(VALIDATE(emp, job_id => '_last'));
```

1 2 Interview-Ready One-Liner (Use this)

“In production, we use COPY INTO with ON_ERROR=CONTINUE to ensure data availability while skipping bad records. We capture rejected rows using the VALIDATE table function for auditing and compliance. VALIDATION_MODE is used in UAT to analyze data quality before loading.”

1 3 Final Cheat Sheet

Need	Use
Stop on first error	ABORT_STATEMENT
Load good data	CONTINUE
Skip bad file	SKIP_FILE
Error preview	VALIDATION_MODE
Store bad rows	VALIDATE
Audit	COPY_HISTORY



```

87 COPY INTO customer_table_internal_stage
88 FROM @DEMO_INTERNAL_STAGE/sampleddata.csv
89 ON_ERROR = CONTINUE
90 VALIDATION_MODE='RETURN_5_ROWS' ;|
91

```

Results			Chart	
	AGE	CUSTOMER_NAME		
1	AGE	CUSTOMERNAME		
2	22	Kelley Alex		
3	44	Finn Aloysius		
4	23	Bryant Ebba		
5	43	Wright Gretchen		